

Laravel RoleMiddleware Explained

How role-based authorization works — OWASP M3 server-side enforcement

1. What This Middleware Does

RoleMiddleware checks whether the authenticated user has one of the roles required by the route. If yes — the request continues. If not — the server returns 403 Forbidden as JSON, and the controller never runs.

This is the OWASP M3 (Insecure Authorization) safeguard — the actual security boundary lives on the server. The Flutter app's role-based UI is just for UX; the server is the only thing that decides whether a request is allowed.

2. The Full Code — `app/Http/Middleware/RoleMiddleware.php`

```
<?php namespace App\Http\Middleware; use Closure; use Illuminate\Http\Request; use
Symfony\Component\HttpFoundation\Response; class RoleMiddleware { public function
handle(Request $request, Closure $next, string ...$roles): Response { if
(!in_array($request->user()?->role, $roles, strict: true)) { return
response()->json(['message' => 'Forbidden'], 403); } return $next($request); } }
```

3. Method Signature Breakdown

```
handle(Request $request, Closure $next, string ...$roles)
```

Parameter	What Laravel Passes	Used For
Request \$request	The incoming HTTP request	Reading \$request->user()
Closure \$next	Callable to continue the pipeline	Calling \$next(\$request) lets the request reach the controller
string ...\$roles	Variadic — comma-split route args	Allowed roles list, e.g. ['admin', 'manager']

How the variadic `...$roles` becomes a list:

```
Route::middleware('role:admin,manager') ■ ▼ Laravel splits the string after ':' by commas
■ ▼ $roles = ['admin', 'manager'] ← inside the middleware
```

4. Step-by-Step Request Flow

Client request: GET /api/admin/users Authorization: Bearer eyJhbGci... (user is 'cashier')

- Request hits `Route::middleware(['auth:api', 'role:admin'])`
- `auth:api` middleware runs FIRST: → `JWTAuth::parseToken()->authenticate()` → verifies signature, exp, blacklist → `User::find(sub)` → sets `$request->user() = User { id: 5, role: 'cashier' }`
- `role:admin` middleware runs SECOND: → `$roles = ['admin']` → `$request->user()?->role = 'cashier'` → `in_array('cashier', ['admin'], strict: true) = false` → returns `response()->json(['message' => 'Forbidden'], 403)`
- Controller NEVER runs – request is stopped here.

If user is admin:

- `role:admin` middleware runs: → `in_array('admin', ['admin'], strict: true) = true` → `return $next($request)` ← passes request along
- Controller runs, returns JSON to client.

5. Why Middleware Order Matters

```
Route::middleware(['auth:api', 'role:admin']) ■ ■ ■ ■ runs SECOND – user is loaded ■ ■  
runs FIRST – validates JWT, sets $request->user()
```

If the order was reversed — role:admin would run before auth:api — \$request->user() would be null because no authentication has happened yet. The null-safe operator ?->role would yield null, in_array would return false, and every request would be rejected with 403, even legitimate admin requests.

6. Two Defensive Details Worth Knowing

a) Why ?-> (null-safe operator)?

```
$request->user()?->role
```

If somehow auth:api was missing from the route (misconfiguration), \$request->user() returns null. Without ?-> calling ->role on null would crash with a TypeError. With ?-> it gracefully returns null. The null then fails the in_array check → 403. Fail-closed by default.

b) Why strict: true in in_array?

```
in_array($request->user()?->role, $roles, strict: true)
```

Without strict mode, PHP's loose comparison can produce dangerous matches:

```
// Without strict – DANGEROUS: in_array(0, ['admin']) = true ← 0 == 'admin' (PHP < 8)  
in_array('1abc', ['1']) = true ← string→int coercion in_array(null, ['', 'admin']) = true  
← null == '' // With strict: true – SAFE: in_array(0, ['admin'], strict: true) = false  
in_array('cashier', ['admin'], strict: true) = false in_array('admin', ['admin'], strict:  
true) = true
```

Without strict mode an attacker submitting role 0 or null could potentially match a string role in older PHP versions. Always use strict: true in security-sensitive comparisons.

7. How It Is Registered as 'role' Alias

In bootstrap/app.php:

```
->withMiddleware(function (Middleware $middleware): void { $middleware->api(prepend: [  
\Illuminate\Http\Middleware\HandleCors::class, ]); $middleware->redirectGuestsTo(fn() =>  
null); $middleware->alias(['role' => \App\Http\Middleware\RoleMiddleware::class]); })
```

alias() maps the short name 'role' to the full class. This is what lets you write middleware('role:admin') in routes instead of middleware(\App\Http\Middleware\RoleMiddleware::class.':admin').

8. Usage Examples in routes/ani.php

```
// Single role Route::middleware(['auth:api', 'role:admin'])->group(function () {  
Route::get('/admin/users', ...); }); // Multiple roles – comma-separated  
Route::middleware(['auth:api', 'role:admin,manager'])->group(function () {  
Route::get('/reports', ...); }); // All roles Route::middleware(['auth:api',  
'role:admin,manager,cashier'])->group(function () { Route::get('/sales', ...); });
```

Route	Allowed Roles	Forbidden Roles
GET /api/admin/users	admin	manager, cashier
DELETE /api/admin/users/{id}	admin	manager, cashier

GET /api/reports	admin, manager	cashier
GET /api/sales	admin, manager, cashier	—
GET /api/auth/me	any authenticated user	guest only

9. OWASP M3 — Why Server-Side Authorization Is Mandatory

OWASP Mobile Top 10 — M3: Insecure Authorization. The flaw is when authorization decisions are made (or only made) on the client.

	Client (Flutter)	Server (Laravel)
Purpose	UX — hide buttons the user cannot use	Security — actually decide what is allowed
Can attacker bypass?	YES — decompile app, modify code, intercept network, don't bother the server	NO — must do it on the server itself
What if client check is missing?	Bad UX — user sees buttons that 403 when clicked	Total breach — anyone can do anything
What if server check is missing?	Looks fine in the app	Catastrophic — no security at all

RoleMiddleware is the server-side check. The Flutter RoleGuard widget is the client-side UX layer. Both are needed — but only the server check is mandatory for security.

Quick Reference Summary

Concept	One-line answer
What is RoleMiddleware?	Server-side check that rejects requests when user role is not in allowed list
When does it run?	After auth:api — needs \$request->user() to be loaded
What does it return on failure?	403 Forbidden as JSON
How are multiple roles passed?	Comma-separated after colon: 'role:admin,manager'
Why ?-> (null-safe)?	Fail-closed if auth:api was misconfigured
Why strict: true in in_array?	Prevents PHP loose-comparison matches (0 == 'admin')
How is 'role' alias registered?	\$middleware->alias(['role' => RoleMiddleware::class]) in bootstrap/app.php
What does OWASP M3 require?	Authorization decisions must be enforced server-side, never trust the client